Lab Week 4 Manual

Memory Visibility, Atomicity & Immutability

Dr. Mohamad AOUDE

May 11, 2026

Course: Concurrency & Distributed Systems (ULFG SEM VIII)
Week: 4 – Memory Visibility, Atomicity & Immutability
Lab: week4-lab / week4-day1-memory-visibility-immutability
Canonical Location: `https://github.com/maoude/concurrency-course-complete`

Learning Objectives

Upon completing this lab, you will:

- Understand the Java Memory Model (JMM) and *visibility* — when writes in one thread become visible to other threads
- Distinguish between *visibility* (memory consistency) and *atomicity* (indivisibility)
- Master `volatile` keyword and *happens-before* relationships
- Recognize and fix *double-checked locking* (DCL) patterns
- Implement *immutable snapshot* patterns for safe concurrent updates
- Use `AtomicReference` and *atomic compare-and-set* (CAS) operations
- Design data structures that avoid visibility races through immutability and indirection

Lab Structure Overview

Part	Topic	Demos	Exercises
1	Visibility	Demo01, Demo02	Ex1
2	Atomicity vs Volatility	Demo03	Ex2
3	Double-Checked Locking	(DCL pattern)	Ex3
4	Immutability	(Snapshot pattern)	Ex4
5	ThreadLocal Preview	Demo04, Demo05	(not graded)

1 Part 1: Memory Visibility

Goal

Understand that memory writes in one thread are not automatically visible to other threads. Synchronization (via locks or `volatile`) establishes happens-before edges that guarantee visibility.

Core Concept: Visibility vs Atomicity

- **Visibility:** When a write to variable X in thread A becomes visible in thread B
- **Atomicity:** Whether a read-modify-write sequence is indivisible (no interleaving)
- **These are orthogonal.** A non-atomic operation can be made visible; an atomic operation can be invisible.

The Java Memory Model (JMM)

Key Rules:

- **Synchronization:** Exiting a synchronized block creates a happens-before with any later entry into a synchronized block on the same lock. Writes inside the block are visible to readers outside.
- **Volatile writes:** A write to a volatile variable happens-before a subsequent read. Volatile variables are bypassed from CPU caches and written to main memory.
- **Final fields:** A write to a final field during object construction happens-before any read of that field (even without volatile or synchronization).
- **Thread creation:** Starting a thread (calling `start()`) happens-before the thread's `run()` method begins.
- **No visibility guarantee without these:** Unsynchronized writes may be cached in CPU registers or L1 cache; other threads may never see them.

Demos

1.0.1 Demo01 – Stale Read Without Volatility

What it demonstrates:

- A `boolean stopFlag` shared between two threads, not volatile
- One thread sets `stopFlag = true`; the other spins in a loop checking `!stopFlag`
- Without volatile, the spinning thread may never see the write (CPU cache stale value)
- The loop may run indefinitely even after the flag is written
- Shows why visibility is necessary, not just atomicity

Key insight: An unsynchronized write in one thread is not guaranteed to be visible to other threads. The JMM allows compiler and CPU optimizations that can hoist reads before loops, caching stale values indefinitely.

1.0.2 Demo02 – Volatility Establishes Visibility

What it demonstrates:

- Same scenario as Demo01, but `stopFlag` is `volatile boolean`
- Write to volatile variable creates a happens-before edge
- Subsequent reads are guaranteed to see the write
- Loop terminates promptly after flag is set
- Cost: volatile reads/writes bypass CPU caches (slower than regular variables)

Code pattern:

```
public class Demo02_StopFlagVolatile {
  private volatile boolean stopFlag = false;  // volatile

  public void spinLoop() {
    while (!stopFlag) {  // Guaranteed to see the write
      // do work
    }
  }

  public void setStop() {
    stopFlag = true;    // Happens-before all subsequent reads
  }
}
```

Exercises

1.0.3 W4.P1.Ex1 – Stop Flag with Volatile

Goal: Fix a visibility race by applying volatile.

Requirements:

- Given a broken `stopFlag` (non-volatile), make it volatile
- Verify that a spinning thread terminates promptly after the flag is set
- Test timing: set flag, measure loop termination time, assert it's reasonable (not indefinite)
- Example: Thread A spins; Thread B sets flag; Assert Thread A exits loop within 1 second

2 Part 2: Atomicity vs Visibility

Goal

Understand that `volatile` provides visibility but *not* atomicity. Compound operations (read-modify-write) need both atomicity and visibility.

Core Concept: Two Separate Concerns

- **Volatile + non-atomic:** Reads and writes are visible, but a sequence like `count++` can still race (read, increment, write are three separate operations).
- **Synchronized + atomic:** Locks provide both atomicity (mutual exclusion) and visibility (happens-before on lock acquire/release).
- **Atomic classes:** `AtomicInteger`, `AtomicReference`, etc. provide atomic operations (e.g., `incrementAndGet()`) with visibility guarantees.

Demos

2.0.1 Demo03 – Volatile Counter is Not Atomic

What it demonstrates:

- A `volatile int counter`
- Two threads increment it many times
- Each read of counter is guaranteed fresh (not cached)
- But `counter++` is still three operations: read, add, write
- Threads race during the read-modify-write window
- Final count is less than expected (lost updates)
- Shows that volatile fixes visibility, not atomicity

Code pattern:

```
public class Demo03_VolatileCounterNotAtomic {
  private volatile int counter = 0;

  public void increment() {
    counter++;  // Volatile reads/writes are visible,
                // but read-modify-write is not atomic
  }
}

// Fix: use synchronized or AtomicInteger
public void incrementAtomic() {
  synchronized(this) {
    counter++;  // Now both visible AND atomic
  }
}
```

Exercises

2.0.2 W4.P2.Ex2 – Atomic Counter

Goal: Implement an atomic counter using `AtomicInteger`.

Requirements:

- Use `AtomicInteger` instead of `volatile int`

- Implement `increment()` using `getAndIncrement()` or `incrementAndGet()`
- Implement `getCount()` using `get()`
- Verify no lost updates under high contention
- Example: Two threads each increment 1,000,000 times; final count must be 2,000,000

Why it works: `AtomicInteger.incrementAndGet()` is a single atomic operation that both reads-modifies-writes (atomicity) and guarantees visibility (happens-before).

3 Part 3: Double-Checked Locking (DCL) & Safe Publication

Goal

Understand the *double-checked locking* pattern, why the naive version is broken, and how `volatile` fixes it. Learn about safe publication and happens-before.

The DCL Pattern (Broken)

A common (incorrect) pattern tries to minimize lock contention:

```
public class BrokenDCL {
  private Helper instance;

  public Helper getInstance() {
    if (instance == null) {  // CHECK (no lock)
      synchronized(this) {
        if (instance == null) {  // CHECK (with lock)
          instance = new Helper();  // INITIALIZE (with lock)
        }
      }
    }
    return instance;
  }
}
```

Why it's broken:

1. **Visibility race:** Even though the synchronized block ensures atomicity of construction, a later thread reading `instance` outside the lock may not see the write (visibility race, not atomicity race).
2. **Reordering race:** The JMM allows reordering of instructions. Construction of `Helper` may not be complete before the reference is assigned to `instance`. A racing thread may see a partially constructed object.

The DCL Pattern (Fixed)

Add `volatile` to the instance field:

```
public class FixedDCL {
  private volatile Helper instance;  // volatile!

  public Helper getInstance() {
    if (instance == null) {  // CHECK (no lock)
      synchronized(this) {
        if (instance == null) {  // CHECK (with lock)
          instance = new Helper();  // Visibility + completion guaranteed
        }
      }
    }
    return instance;
  }
}
```

Why it works:

- **Volatile write:** The assignment to `instance` is a volatile write, creating a happens-before with any subsequent volatile read.
- **Construction visibility:** All writes during `Helper()` construction happen-before the volatile write to `instance`.
- **Safe publication:** Any thread that sees the non-null `instance` is guaranteed to see a fully constructed object.

Exercises

3.0.1 W4.P3.Ex3 – DCL Singleton with Volatile

Goal: Implement a thread-safe singleton using double-checked locking with volatile.

Requirements:

- Declare `private static volatile Singleton instance`
- Implement `public static Singleton getInstance()`
- First check: `if (instance == null)` outside lock
- Second check: inside synchronized block
- Constructor must be private
- Verify multiple threads see the same instance (even under first-call races)

Hint: Without volatile, threads may see stale `null` or partially constructed objects. Volatile ensures visibility and prevents reordering during construction.

4 Part 4: Immutability & Atomic Reference

Goal

Understand how *immutability* eliminates visibility races and enables safe concurrent access without locks. Learn the *atomic snapshot update* pattern using `AtomicReference`.

Core Concept: Immutability as Thread Safety

Key insight: If an object is immutable (all fields final, no mutations after construction), then any thread can safely read it without synchronization. Visibility of the object reference is all that's needed, not locking the fields.

Immutable snapshot pattern:

1. Create an immutable snapshot object (e.g., a record or class with all final fields)
2. Store a reference to the current snapshot in an `AtomicReference`
3. To update: create a new immutable snapshot and use `compareAndSet()` to swap references
4. All reads are lock-free (just dereference the atomic reference); updates are nonblocking CAS

Demos

4.0.1 Immutable Snapshot Cache

What it demonstrates:

- An immutable record (or final-field class) representing a cache snapshot
- `AtomicReference<Snapshot>` holds the current snapshot
- Readers use `atomicRef.get()` to fetch the snapshot (very fast, no lock)
- Writers create a new snapshot and use `compareAndSet()` to swap (atomic compare-and-swap)
- No locks, no BLOCKED threads, excellent scalability for read-heavy workloads

Code pattern:

```
public class ImmutableSnapshotCache<K, V> {
  record Snapshot<K, V>(Map<K, V> data) {}

  private AtomicReference<Snapshot<K, V>> snapshot =
    new AtomicReference<>(new Snapshot<>(Map.of()));

  public V get(K key) {
    // Lock-free read
    return snapshot.get().data().get(key);
  }

  public void put(K key, V value) {
    // Atomic swap via CAS
    Snapshot<K, V> current = snapshot.get();
    Map<K, V> newData = new HashMap<>(current.data());
    newData.put(key, value);
    snapshot.compareAndSet(current,
                           new Snapshot<>(newData));
  }
}
```

Exercises

4.0.2 W4.P4.Ex4 – Immutable Snapshot Cache

Goal: Implement a thread-safe cache using immutable snapshots and atomic reference.

Requirements:

- Create an immutable `Snapshot<K, V>` record (or class with final fields)
- Use `AtomicReference<Snapshot>` to store the current snapshot
- Implement `get(K key)` as a lock-free read
- Implement `put(K key, V value)` using `compareAndSet()`:
  1. Read current snapshot
  2. Create new HashMap from current data
  3. Add the new key-value pair
  4. Create new immutable snapshot
  5. Use `compareAndSet()` to swap (loop if CAS fails due to concurrent update)
- Verify no data loss under concurrent reads and writes
- Example: 100 concurrent readers + 10 writers; final cache has all written keys

Tradeoff: Snapshot creation (copying the map) has CPU cost on writes, but reads are completely lock-free. Ideal for read-heavy workloads.

5 Part 5: ThreadLocal Preview (Week 5 Material, Enrichment Only)

Context

This part previews Week 5 material to connect memory visibility and shared-state problems to thread-local storage. These demos are *not* part of the graded Week 4 exercise set.

Demos

5.0.1 Demo04 – Unsafe Shared Date

What it demonstrates:

- Multiple threads share a single `SimpleDateFormat` instance
- `SimpleDateFormat` is not thread-safe (internal state mutation)
- Concurrent calls corrupt the formatter and produce incorrect results
- Shows why thread-unsafe utility classes must not be shared

5.0.2 Demo05 – ThreadLocal Safe Date

What it demonstrates:

- Each thread gets its own `SimpleDateFormat` via `ThreadLocal`
- No sharing, no synchronization needed
- Each thread sees its own copy; no visibility or atomicity races
- Format operations are safe because they're not concurrent

Code pattern:

```
public class ThreadLocalDateFormatter {
  private static final ThreadLocal<SimpleDateFormat> df =
    ThreadLocal.withInitial(() ->
      new SimpleDateFormat("yyyy-MM-dd"));

  public String format(Date date) {
    return df.get().format(date);  // Thread's own copy
  }
}
```

Key Takeaways

1. **Visibility is separate from atomicity.** Volatile provides visibility without atomicity; locks provide both.
2. **The Java Memory Model defines happens-before relationships.** Synchronization, volatile, and final fields establish these relationships.
3. **Volatile writes establish visibility.** A write to a volatile variable happens-before any subsequent read.
4. **Compound operations need atomicity.** Read-modify-write sequences race even with volatile; use AtomicXXX or locks.
5. **Double-checked locking is broken without volatile.** Visibility and reordering races can expose partially constructed objects.
6. **Immutability enables lock-free concurrency.** If an object is immutable, reads need no synchronization.
7. **Atomic references with immutable snapshots scale well.** Reads are lock-free; writes use CAS (nonblocking).
8. **Safe publication** means all threads see a fully constructed object. Volatile fields, final fields, and lock handoffs ensure this.

Next Steps:

- Complete all 4 graded exercises (Ex1, Ex2, Ex3, Ex4)
- Run `./gradlew studentCheck` to validate
- (Optional) Explore Demo04 and Demo05 as ThreadLocal preview
- Review `PITFALLS.md` for memory model pitfalls
- Week 5 focuses on ThreadLocal depth, java.util.concurrent APIs, and backpressure

Resources

GitHub Repository: `https://github.com/maoude/concurrency-course-complete`
LinkedIn Profile: `https://www.linkedin.com/in/mohamadaoude/`

GitHub LinkedIn